



**National Teachers College**

629 J Nepomuceno, Quiapo, Manila, 1001 Metro Manila  
Bachelor of Science in Information Technology

***STRAKTYUR***  
***SDG 3: Good Health and Well-being***  
***MediTrack: Intelligent Medicine Management System***

**A Final Project on DATA STRUCTURE**  
College of Information Technology

In partial fulfillment  
of the Course Requirements for the degree  
of Bachelor of Science in Information Technology

**Submitted by:**

*Bermio, Carl Jann*

*Gonzales, Joshua*

*Malda, Kenneth*

*Nepa, Jastine*

*Pajo, Harvey*

*2.2 BSIT*

*National Teacher College*

*Justine Louise Neypes*

*December 13, 2025*

# TABLE OF CONTENTS

|                                                                     | Page         |
|---------------------------------------------------------------------|--------------|
| <b>I. INTRODUCTION</b>                                              |              |
| <i>1.1. Project Overview &amp; UN SDG Target</i>                    | <b>2</b>     |
| <i>1.2. Problem Statement</i>                                       | <b>3-4</b>   |
| <br><b>II. REQUIREMENTS &amp; ANALYSIS</b>                          |              |
| <i>2.1. Functional Requirements and Non-Functional Requirements</i> | <b>5-9</b>   |
| <i>2.2. Data Requirements</i>                                       | <b>10-11</b> |
| <i>2.3. Complexity Analysis: Expected Time</i>                      | <b>12</b>    |
| <i>Space complexity of the Core Algorithm</i>                       |              |
| <br><b>III. DESIGN SPECIFICATION</b>                                |              |
| <i>3.1. Core Data Structures Used (The Five)</i>                    | <b>13-14</b> |
| <i>3.2. Algorithm Flowchart</i>                                     | <b>15-16</b> |
| <i>3.3. Module Breakdown</i>                                        | <b>16-20</b> |
| <br><b>IV. TESTING &amp; RESULTS</b>                                |              |
| <i>4.1. Test Cases</i>                                              | <b>21</b>    |
| <i>4.2. Performance Test</i>                                        | <b>22</b>    |
| <br><b>V. CONCLUSION &amp; CONTRIBUTION</b>                         |              |
| <i>5.1. Conclusion</i>                                              | <b>23</b>    |
| <i>5.2. Individual Contributions</i>                                | <b>23-24</b> |

# INTRODUCTION

## ***Project Overview & UN SDG Target***

MediTrack is an all-encompassing digital platform for medication management, developed with a focus on overcoming non-compliance with medication using smart scheduling, smart reminders, and tracking. MediTrack encourages patients to take control of their medication regimes, get context-sensitive reminders, and keep track of accurate medication history using efficient algorithms and data structures. With a blend of healthcare technology and usability, MediTrack fills a gap in fulfilling medical prescriptions and actual intake of medication using technology for better health outcomes.

## ***UN SDG Target - SDG 3: Good health and Well-being***

How MediTrack Contributes to SDG 3: Good Health and Well-being

MediTrack helps improve health outcomes in several ways:

- **Prevents Disease Complications:** By ensuring patients take their medications as prescribed, MediTrack helps prevent complications from chronic diseases, reducing hospital admissions and premature deaths.
- **Reduces Healthcare Costs:** Non-adherence costs the healthcare system billions each year due to preventable hospitalizations and complications. MediTrack addresses this by improving medication adherence and reducing the economic burden on healthcare systems.
- **Improves Quality of Life:** By empowering patients to take control of their treatment plans, MediTrack helps improve long-term health outcomes and the overall quality of life for patients with chronic conditions.

## ***Problem Statement***

### ***Current Problem:***

- ❖ Medication non-adherence is a huge issue in healthcare, it mainly affects the elderly, people with chronic conditions, and those recovering from surgery. Studies show that around 50-60% of patients with chronic conditions don't take their medications as prescribed.
- ❖ It costs \$100-300 billion in the healthcare system annually due to hospital readmissions, complications, and untreated conditions.
- ❖ For many patients, especially in developing countries, sticking to a medication schedule is tough. Although there are basic solutions being offered like pillboxes or paper reminders exist, they don't address the complexity of managing multiple medications or provide enough support to help patients stay on track long-term.
- ❖ Forgetfulness and lack of routine affects especially the elderly, simply forgetting to take their medications is a major challenge because without reminders or a structured routine, missed doses are inevitable, leading to lack of adherence over time.

### ***Proposed Solution:***

To provide a solution for the problem of medication non-adherence, MediTrack offers an all-in-one platform with the following key features:

#### **→ Smart medication scheduling and reminders**

- Personalized schedules: MediTrack allows patients to input their medication schedules, including the time, dosage, and frequency. It ensures that even if the regimen is complex, the patient gets clear, easy-to-follow instructions.

- Context-Aware Alerts: Instead of just standard reminders, MediTrack uses intelligent reminders based on time of day, patient routines, and lifestyle habits.
- Multimedia reminders: Users receive reminders through screen notifications.

#### → **Medication Tracking**

- Digital History: MediTrack stores a detailed history of the medications taken, including dates, dosages, and times. This helps patients keep track of their medication intake and share the information with their healthcare provider if needed.

#### → **Integration with Healthcare Providers**

- Health Provider Dashboards: Healthcare providers can access a secure dashboard to monitor their patients' adherence in real time. This helps doctors adjust treatment plans if necessary and intervene early if adherence is an issue.
- Prescription Reminders: MediTrack can send reminders to patients and doctors for medication refills to ensure there's no gap in treatment.

## REQUIREMENTS & ANALYSIS

### *Functional Requirements and Non-Functional Requirements*

#### FUNCTIONAL REQUIREMENTS

| ID         | REQUIREMENTS               | DESCRIPTION                                                                                                                                                                                                                                       |
|------------|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>FR1</i> | <i>CREATING AN ACCOUNT</i> | <ul style="list-style-type: none"><li>• Users can sign up for a new account</li><li>• Login information gets stored in a file called account.txt</li><li>• The system won't allow two accounts with the same username</li></ul>                   |
| <i>FR2</i> | <i>SIGNING IN</i>          | <ul style="list-style-type: none"><li>• Users can log in with their username and password</li><li>• The system checks if the login details match what's saved in account.txt</li><li>• Login fails if the username or password is wrong</li></ul> |
| <i>FR3</i> | <i>SIGNING OUT</i>         | <ul style="list-style-type: none"><li>• Users can log out when they're done</li><li>• After logging out, users return to the login screen</li></ul>                                                                                               |

|                   |                                         |                                                                                                                                                                                                                                                                                                                                                         |
|-------------------|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>FR4</i></b> | <b><i>MAIN-MENU OPTIONS</i></b>         | <ul style="list-style-type: none"> <li>• After logging in, users see a menu with these choices:             <ol style="list-style-type: none"> <li>1. Add new medicine</li> <li>2. View medicines</li> <li>3. Mark medicines as taken</li> <li>4. Delete Medicine</li> <li>5. Reset Medicine</li> <li>6. Logout</li> </ol> </li> </ul>                  |
| <b><i>FR5</i></b> | <b><i>ADDING MEDICINES</i></b>          | <ul style="list-style-type: none"> <li>• Users can add medicines by entering:             <ul style="list-style-type: none"> <li>○ Name of the medicine</li> <li>○ How much to take</li> <li>○ What time to take it</li> </ul> </li> <li>• Medicine details are saved in medicine.txt</li> <li>• New medicines start with a "Pending" status</li> </ul> |
| <b><i>FR6</i></b> | <b><i>VIEWING SCHEDULE</i></b>          | <ul style="list-style-type: none"> <li>• Users can see all their scheduled medicines</li> <li>• The information comes from the medicine.txt file</li> </ul>                                                                                                                                                                                             |
| <b><i>FR7</i></b> | <b><i>RECORDING MEDICINE INTAKE</i></b> | <ul style="list-style-type: none"> <li>• Users can mark whether they took their medicine or not</li> </ul>                                                                                                                                                                                                                                              |

|             |                            |                                                                                                                                                                                                                                                                                            |
|-------------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|             |                            | <ul style="list-style-type: none"> <li>• The status updates and saves to medicine.txt</li> </ul>                                                                                                                                                                                           |
| <b>FR8</b>  | <b>REMOVING MEDICINES</b>  | <ul style="list-style-type: none"> <li>• Users can delete a medicine by typing its name</li> <li>• The medicine gets permanently removed from medicine.txt</li> </ul>                                                                                                                      |
| <b>FR9</b>  | <b>DAILY STATUS RESET</b>  | <ul style="list-style-type: none"> <li>• All medicine statuses automatically reset to "Pending" when users select option 5</li> </ul>                                                                                                                                                      |
| <b>FR10</b> | <b>REMINDERS</b>           | <ul style="list-style-type: none"> <li>• The system constantly monitors the schedule while users logged in</li> <li>• Users get notified when it's time to take their medicine</li> <li>• Users will be ask after the three notification pop up if he already take the medicine</li> </ul> |
| <b>FR11</b> | <b>CLOSING THE PROGRAM</b> | <ul style="list-style-type: none"> <li>• Users can safely exit the application</li> <li>• All background processes stop when exiting</li> </ul>                                                                                                                                            |



## NON-FUNCTIONAL REQUIREMENTS

| ID          | REQUIREMENTS          | DESCRIPTION                                                                                                                                                                                                                      |
|-------------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>NFR1</i> | <i>EASY TO USE</i>    | <ul style="list-style-type: none"><li>• Simple menu-based interface anyone can understand</li><li>• Clear instructions guide users through each action</li></ul>                                                                 |
| <i>NFR2</i> | <i>SPEED</i>          | <ul style="list-style-type: none"><li>• Medicine schedules are checked immediately</li><li>• Logging in and switching menus takes 1-2 seconds or less</li></ul>                                                                  |
| <i>NF3</i>  | <i>DEPENDABILITY</i>  | <ul style="list-style-type: none"><li>• The system saves and loads data correctly from files</li><li>• User information and medicine data don't get lost during normal use</li></ul>                                             |
| <i>NFR4</i> | <i>EASY TO UPDATE</i> | <ul style="list-style-type: none"><li>• Code is organized into separate sections for:<ul style="list-style-type: none"><li>○ Login/logout features</li><li>○ Medicine management</li><li>○ Scheduling system</li></ul></li></ul> |

|                    |                                    |                                                                                                                                                                                             |
|--------------------|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><i>NFR5</i></b> | <b><i>SYSTEM COMPATIBILITY</i></b> | <ul style="list-style-type: none"> <li>• Works on Windows computers with a C++ compiler</li> <li>• Doesn't need internet connection or database software</li> </ul>                         |
| <b><i>NFR6</i></b> | <b><i>GROWTH CAPABILITY</i></b>    | <ul style="list-style-type: none"> <li>• Can handle multiple users through file storage</li> <li>• Can manage growing amounts of medicine records without problems</li> </ul>               |
| <b><i>NFR7</i></b> | <b><i>ALWAYS RUNNING</i></b>       | <ul style="list-style-type: none"> <li>• Reminder checking continues as long as the app is open</li> <li>• Notifications appear even when users aren't actively using the system</li> </ul> |
| <b><i>NFR8</i></b> | <b><i>DATA ACCURACY</i></b>        | <ul style="list-style-type: none"> <li>• Changes to medicine.txt and account.txt are saved correctly</li> <li>• Prevents incomplete or corrupted file saves</li> </ul>                      |

## ***Data Requirements***

MediTrack uses two primary text files for data persistence, employing a pipe-separated value format for structured storage. The system maintains data in plain text files to ensure simplicity, portability, and easy manual inspection.

### **USER DATA FILE**

```
FullName|Age|Username|Password
```

#### ❖ Field Details:

- FullName: User's complete name (alphabetic characters and spaces)
- Age: User's age stored as string (numeric validation 1-150)
- Username: Unique login identifier (alphanumeric)
- Password: Authentication credential (plain text for simplicity)

#### ❖ Size Specifications:

- Maximum line length: ~100 characters
- Average record size: 80-120 bytes
- Maximum capacity: 1,000 users
- Total file size: Approximately 100 KB

### **MEDICINE DATA FILE:**

```
UserID|MedicineName|Dose|Time|Status
```

#### ❖ Field Details:

- UserID: Reference to the user who owns this medication (matches Username from users.txt)
- MedicineName: Name of the medication (descriptive text)
- Dose: Dosage instructions (text description)
- Time: Scheduled time in 24-hour HH:MM format
- Status: Current state - either "Pending" or "Taken"

❖ Size Specifications:

- Maximum line length: ~150 characters
- Average record size: 130-180 bytes
- Maximum capacity: 10,000 medicine records
- Total file size: Approximately 1.8 MB

## ***Complexity Analysis: Expected Time/ Space complexity of the Core Algorithm***

### **LINKED LIST**

| <b>OPERATIONS</b> | <b>TIME COMPLEXITY</b>                   | <b>EXPLANATION</b>                                           |
|-------------------|------------------------------------------|--------------------------------------------------------------|
| Insert            | <b>O(1)</b> (if using head/tail pointer) | Fast, just reassign pointers                                 |
| Delete            | <b>O(1)</b> (given pointer to the node)  | No need to shift elements                                    |
| Search            | <b>O(n)</b>                              | Acceptable because medicine lists per user are usually small |

### **HASH TABLES**

| <b>OPERATIONS</b> | <b>AVERAGE CASE</b> | <b>EXPLANATION</b>                     |
|-------------------|---------------------|----------------------------------------|
| Insert            | <b>O(1)</b>         | Stores user or medicine data instantly |
| Search            | <b>O(1)</b>         | Quick login and fast medicine lookup   |
| Delete            | <b>O(1)</b>         | Helps when removing old reminders      |

### **SORTING ALGORITHMS**

| <b>OPERATIONS</b> | <b>TIME COMPLEXITY</b> | <b>EXPLANATION</b>               |
|-------------------|------------------------|----------------------------------|
| Merge Sort        | <b>O(n log n)</b>      | Stable, good for time-based data |
| Quick Sort        | <b>O(n log n)</b> avg  | Very fast for large lists        |

## DESIGN SPECIFICATIONS

### *Core Data Structures Used (The Five)*

#### ❖ **Linked List (Storing Medicine Data)**

We used Linked list in storing medicine lists, so the user can easily fill the medicine they want to add like name,dose,time,status pointer to next medicine. We also used a linked list in identifying which medicine status is pending so if the user selects option 3 which is to mark medicine as taken the first medicine will be asked if it's already taken is the medicine that is pending and who is in the line first, sorting them by their schedule time. Using a linked list it's more efficient because no shifting will happen, the medicine list will not be disturbed

```
struct Medicine {
    string name;
    string dose;
    string time;
    string status;
    Medicine* next;
};

Medicine* m = new Medicine{name, dose, time, "Pending", nullptr};
Medicine*& head = medicineTable[userID];
if (!head || head->time > time) {
    m->next = head;
    head = m;
} else {
    Medicine* curr = head;
    while (curr->next && curr->next->time <= time)
        curr = curr->next;
    m->next = curr->next;
    curr->next = m;
```

```
}
```

#### ❖ Hash Tables (User authentication, Medicine reminder tracking, Identifying the userID when adding medicine)

In our research a hash table is good for retrieving data at a constant time. This will help us to authenticate the users credentials by checking if their username and password that they input is existing in our logs. Also for checking the schedule of the medicine that users need to take, so if users need to take medicine at 11:59, using a hash table it will quickly check if there's a schedule for this medicine. Lastly we also used hash by identifying the key value of the user when adding medicines.

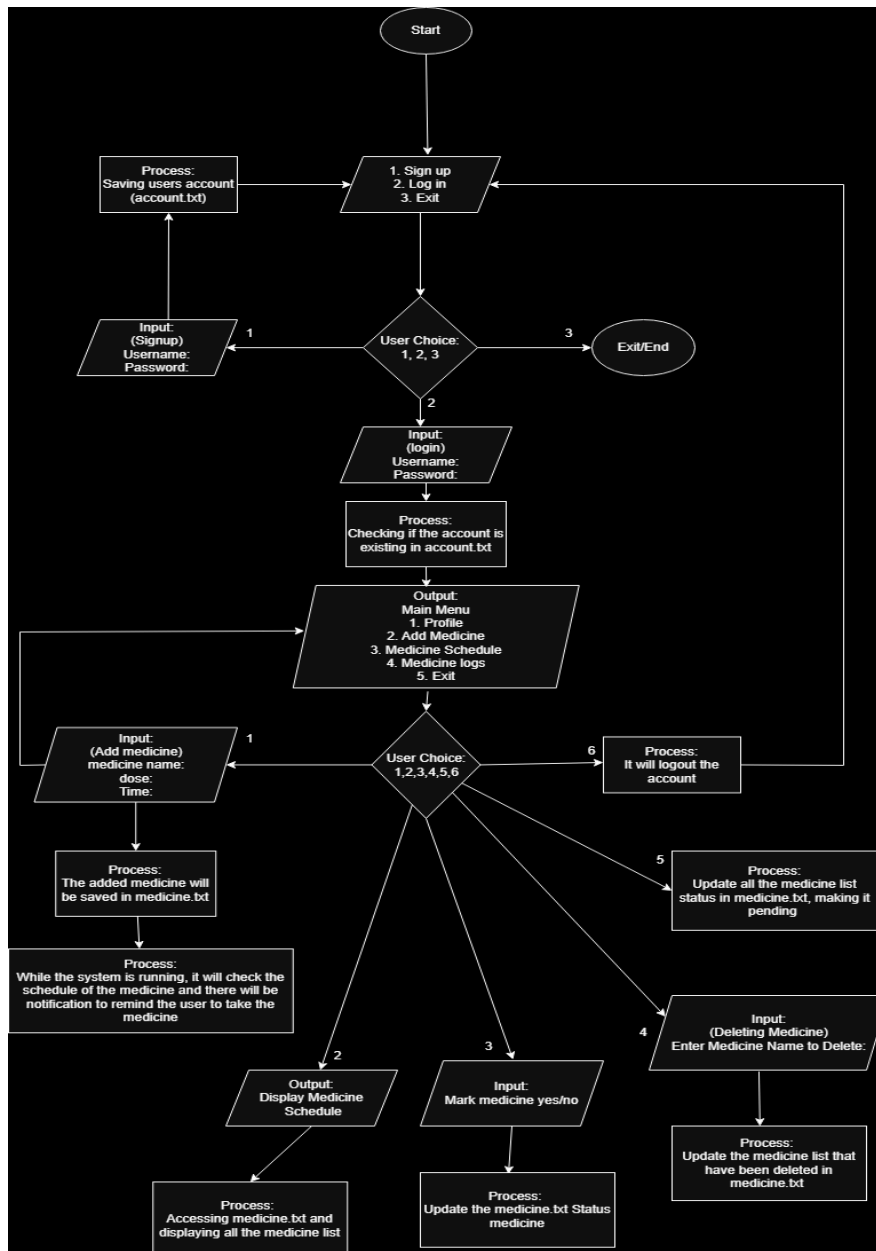
```
unordered_map<string, User> userTable;  
unordered_map<string, Medicine*> medicineTable;  
unordered_map<string, Medicine*> medicineHash;  
unordered_map<string, int> reminderCount;
```

#### ❖ Sorting Algorithms (Organizing Medicine by Time)

This will help users to organize medicine schedules in the right order. The medicine schedule list will be sorted based on their time schedule. By applying this algorithm, users can easily identify what is next after he take medicine.

```
while (curr->next && curr->next->time <= m->time)  
    curr = curr->next;
```

## Algorithm Flowchart



The system begins by loading all registered user data and displaying the main menu, where users can register, log in, or exit the program. During registration, user details are saved to a file, while login verifies credentials using a hash table for fast access.

After a successful login, the system starts a background reminder thread and displays the user menu. From this menu, users can add medicines, view their medicine schedule, mark medicines



as taken, delete medicines, or reset all medicines to pending status. Medicines are stored per user in a linked list and are automatically sorted by time.

While the user interacts with the system, the reminder thread runs in parallel, continuously checking the current time. When a scheduled medicine time is reached and the medicine is still pending, the system displays reminder pop-up notifications and limits them to a fixed number to avoid spamming.

When the user logs out, the reminder thread is safely stopped and control returns to the main menu. Choosing exit terminates the program cleanly. Overall, the flowchart represents a user-driven medicine management system with persistent storage, real-time reminders, and safe concurrent processing.

## ***Module Breakdown***

### **USER REGISTRATION FLOW:**

```
main.cpp
  ↓ calls
registerUser()      // User.cpp
  ↓ uses
userTable (hash map) // Check uniqueness
  ↓ writes
users.txt           // File I/O
  ↓ returns to
main.cpp            // Continue to login
```

**Purpose:** This flow handles new user registration, ensuring that each user ID is unique, and saves user information for future logins.

#### **main.cpp**

- Acts as the starting point of the program.
- Calls the registerUser() function when a user chooses to create an account.

#### **registerUser() – (User.cpp)**

- Collects user details such as user ID and password.
- Checks if the user ID already exists to prevent duplicate accounts.

#### **userTable (Hash Map)**

- Stores registered users in memory.
- Allows fast lookup to verify whether a user ID is already taken.

#### **users.txt (File I/O)**

- If the user ID is unique, the user's information is saved to users.txt.
- This ensures user data is persistent even after the program is closed.

#### **Return to main.cpp**

- After successful registration, control goes back to main.cpp.
- The program proceeds to the login process.

#### **MEDICINE MANAGEMENT FLOW:**

```

userMenu()          // main.cpp
  ↓ calls
addMedicine()       // MedicineList.cpp
  ↓ accesses
medicineTable[userID] // Hash table lookup
  ↓ manipulates
Linked List         // Sorted insertion
  ↓ calls
saveMedicines()     // Write to medicine.txt
  ↓ triggers
loadMedicines()     // In reminder thread

```

**Purpose:** This flow manages adding, organizing, and saving medicines for a specific user.

#### **userMenu() – (main.cpp)**

- Displays options for logged-in users.

- Calls addMedicine() when the user chooses to add a new medicine.

#### **addMedicine() – (MedicineList.cpp)**

- Collects medicine details such as name and time.
- Adds the medicine to the user's medicine list.

#### **medicineTable[userID] (Hash Table Lookup)**

- Retrieves the medicine list associated with the currently logged-in user.
- Each user has their own independent list of medicines.

#### **Linked List (Sorted Insertion)**

- Medicines are stored in a linked list.
- New medicines are inserted in sorted order (usually by time) for easier processing and reminders.

#### **saveMedicines()**

- Saves all medicine data to medicine.txt.
- Ensures medicine information is not lost when the program exits.

#### **loadMedicines() (Triggered in Reminder Thread)**

- Reloads the updated medicine list.
- Keeps the reminder system synchronized with the latest data.

#### **REMINDER SYSTEM FLOW:**

```
startReminderThread() // MedicineReminder.cpp
↓ creates
Background Thread    // Runs reminderLoop()
↓ calls
getCurrentTimeHHMM() // ReminderTime.cpp
↓ compares with
medicineTable[userID] // Linked list traversal
```

```

↓ if match
makeKey()           // Create reminder key
↓ checks
reminderCount[key]  // Track notifications
↓ if <3
MessageBoxA()       // Windows notification

```

**Purpose:** This flow runs in the background and notifies users when it is time to take their medicine.

### **startReminderThread() – (MedicineReminder.cpp)**

- Starts a separate background thread.
- Allows reminders to run continuously without interrupting the main program.

### **Background Thread (reminderLoop())**

- Continuously checks the current time.
- Runs independently of user input.

### **getCurrentTimeHHMM() – (ReminderTime.cpp)**

- Retrieves the current system time in HH:MM format.
- Used for time comparison with scheduled medicine times.

### **medicineTable[userID] (Linked List Traversal)**

- Loops through the user's medicine list.
- Compare each medicine's scheduled time with the current time.

### **If Time Matches**

- The system detects that it is time to take a medicine.

### **makeKey()**

- Generates a unique key for the reminder.
- Helps prevent excessive notifications for the same medicine.

#### **reminderCount[key]**

- Tracks how many times a notification has been sent.
- Limits notifications to avoid spamming the user.

#### **If reminderCount < 3**

- The system allows the notification to be shown.

#### **MessageBoxA() (Windows Notification)**

- Displays a pop-up reminder message on the screen.
- Alerts the user to take their medicine.

## TESTING & RESULTS

### Test Cases

| Test Case ID | Feature Tested               | Preconditions                                                                                           | Test Steps                                                                                                                                            | Expected Result                                                                                                              |
|--------------|------------------------------|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| TC 1         | Successful Medication Intake | User is logged in. A medicine (Ex. Paracetamol) is added with a status of "Pending" and a current time. | 1. Wait for the system to display the final confirmation box (after 3 reminders).<br>2. Select 'Yes'.<br>3. View Medicines.                           | The status of "Aspirin" should successfully transition from "Pending" to "Taken."                                            |
| TC 2         | Non-Adherence/Missed Intake  | User is logged in. A medicine (Ex. Paracetamol) is added with a status of "Pending" and a current time. | 1. Wait for the system to display the final confirmation box (after 3 reminders).<br>2. Select 'No'.<br>3. View Medicines.                            | The status of "Paracetamol" should remain as "Pending" to track the missed dose.                                             |
| TC 3         | Successful Medicine Addition | User is logged in.                                                                                      | 1. Choose Option<br><b>1. Add Medicine.</b><br>2. Enter valid inputs for Medicine Name, Dose, and Time.<br>3. Choose Option <b>2. View Medicines.</b> | The newly added medicine should appear correctly in the user's list with the initial status of "Pending."                    |
| TC 4         | Invalid Deletion Attempt     | User is logged in. The medicine list contains "Vitamin C."                                              | 1. Choose Option <b>4. Delete Medicine.</b><br>2. Enter the name of a medicine that does not exist (Ex. Mefenamic).                                   | The system should display an error message (Ex. Invalid choice.) and the existing list of medicines should remain unchanged. |

```

C:\Users\Jairon Gonzales\Dov x + ~
1. Add Medicine
2. View Medicines
3. Mark Medicine as Taken
4. Delete Medicine
5. Reset Medicines
6. Logout
Choose: 2
12:00 - Biogesic | Dose: 500mg | Status: Pending
14:01 - Paracetamol | Dose: 20mg | Status: Pending
14:02 - Ibuprofen | Dose: 20mg | Status: Pending
14:03 - Aspirin | Dose: 20mg | Status: Pending
14:04 - Amoxicillin | Dose: | Status: Pending
14:05 - Azithromycin | Dose: 20mg | Status: Pending
14:06 - Cetirizine | Dose: 20mg | Status: Pending
14:07 - Cetirizine | Dose: 20mg | Status: Pending
14:08 - Loratadine | Dose: 20mg | Status: Pending
14:09 - Diphenhydramine | Dose: 20mg | Status: Pending
14:10 - Omeprazole | Dose: 20mg | Status: Pending
14:11 - Pantoprazole | Dose: 20mg | Status: Pending
14:12 - Metformin | Dose: 20mg | Status: Pending
14:13 - Amlodipine | Dose: 20mg | Status: Pending
14:14 - Losartan | Dose: 20mg | Status: Pending
14:15 - Simvastatin | Dose: 20mg | Status: Pending
14:16 - Salbutamol | Dose: 20mg | Status: Pending
14:17 - Prednisone | Dose: 20mg | Status: Pending
14:18 - Hydrocortisone | Dose: 20mg | Status: Pending
14:19 - Loperamide | Dose: 20mg | Status: Pending
14:20 - Oral Rehydration Salts | Dose: 20mg | Status: Pending
15:11 - paracetamol | Dose: 500mg | Status: Taken
  
```

## Performance Test

```
=== PERFORMANCE TEST (60 Records) ===
Testing NFR1: All operations under 2 seconds

08:20 - Medicine_60 | Dose: 1 pill(s) | Status: Pending
08:22 - Medicine_12 | Dose: 1 pill(s) | Status: Pending
08:34 - Medicine_24 | Dose: 1 pill(s) | Status: Pending
08:46 - Medicine_36 | Dose: 1 pill(s) | Status: Pending
08:58 - Medicine_48 | Dose: 1 pill(s) | Status: Pending
09:11 - Medicine_1 | Dose: 2 pill(s) | Status: Pending
09:23 - Medicine_13 | Dose: 2 pill(s) | Status: Pending
09:35 - Medicine_25 | Dose: 2 pill(s) | Status: Pending
09:47 - Medicine_37 | Dose: 2 pill(s) | Status: Pending
09:59 - Medicine_49 | Dose: 2 pill(s) | Status: Pending
10:10 - Medicine_50 | Dose: 3 pill(s) | Status: Pending
10:12 - Medicine_2 | Dose: 3 pill(s) | Status: Pending
10:24 - Medicine_14 | Dose: 3 pill(s) | Status: Pending
10:36 - Medicine_26 | Dose: 3 pill(s) | Status: Pending
10:48 - Medicine_38 | Dose: 3 pill(s) | Status: Pending
11:11 - Medicine_51 | Dose: 1 pill(s) | Status: Pending
11:13 - Medicine_3 | Dose: 1 pill(s) | Status: Pending
11:25 - Medicine_15 | Dose: 1 pill(s) | Status: Pending
11:37 - Medicine_27 | Dose: 1 pill(s) | Status: Pending
11:49 - Medicine_39 | Dose: 1 pill(s) | Status: Pending
12:12 - Medicine_52 | Dose: 2 pill(s) | Status: Pending
12:14 - Medicine_4 | Dose: 2 pill(s) | Status: Pending
12:26 - Medicine_16 | Dose: 2 pill(s) | Status: Pending
12:38 - Medicine_28 | Dose: 2 pill(s) | Status: Pending
12:50 - Medicine_40 | Dose: 2 pill(s) | Status: Pending
13:13 - Medicine_53 | Dose: 3 pill(s) | Status: Pending
13:15 - Medicine_5 | Dose: 3 pill(s) | Status: Pending

18:20 - Medicine_10 | Dose: 2 pill(s) | Status: Pending
18:32 - Medicine_22 | Dose: 2 pill(s) | Status: Pending
18:44 - Medicine_34 | Dose: 2 pill(s) | Status: Pending
18:56 - Medicine_46 | Dose: 2 pill(s) | Status: Pending
19:19 - Medicine_59 | Dose: 3 pill(s) | Status: Pending
19:21 - Medicine_11 | Dose: 3 pill(s) | Status: Pending
19:33 - Medicine_23 | Dose: 3 pill(s) | Status: Pending
19:45 - Medicine_35 | Dose: 3 pill(s) | Status: Pending
19:57 - Medicine_47 | Dose: 3 pill(s) | Status: Pending

=== TEST RESULTS ===
1. Load Medicines (60 records): 15 ms
2. View Medicines (60 records): 23 ms
3. Add + Save Medicine: 0 ms
4. Save Medicines (61 records): 0 ms
=====

=== NFR1 VALIDATION ===
Γ£à Load Medicines: 15 ms < 2000 ms
Γ£à View Medicines: 23 ms < 2000 ms
Γ£à Add Medicine: 0 ms < 2000 ms
Γ£à Save Medicines: 0 ms < 2000 ms
=====
≡fÄë NFR1 PASSED: All operations complete within 2 seconds!

=== STRESS TEST (500 Records) ===
Load 500 records: 27 ms
Γ£à Stress Test PASSED: 500 records loaded in 27 ms (< 2000 ms)

Press Enter to exit...|
```

## CONCLUSION & CONTRIBUTION

### *Conclusion*

MediTrack is an innovative solution to the problem of medication non-adherence. By offering personalized reminders, real-time tracking, and support for both patients and healthcare providers, MediTrack can significantly improve medication adherence, leading to better health outcomes and reduced healthcare costs. This aligns with the UN's SDG 3 goal of ensuring good health and well-being for all, especially in preventing and treating non-communicable diseases.

### *Individual Contributions*

#### **I. Introduction**

1.1. Project Overview & UN SDG Target - **PAJO, HARVEY**

1.2. Problem Statement (What real-world problem does the app solve?) - **BERMIO, CARL**

#### **II. Requirements & Analysis**

2.1. Functional Requirements and Non-Functional Requirements (List of features, e.g., FR1, FR2) - **NEPA, JASTINE**

2.2. Data Requirements (Description of input data structure and size) - **PAJO, HARVEY**

2.3. Complexity Analysis: Expected Time/Space complexity of the Core Algorithm (justify using Big O notation).- **NEPA, JASTINE**

#### **III. Design Specification**

3.1. Core Data Structures Used (The Five): **NEPA, JASTINE**

For each of the five required DSA concepts, include a section detailing:

Justification: Why was this specific DSA chosen for its role?

Implementation Details: How did you implement it (e.g., adjacency list for Graph, array for Heap)?



3.2. Algorithm Flowchart: Include the Flowchart for the system's most complex function (the core algorithm using a Finals concept). **NEPA, JASTINE**

3.3. Module Breakdown: Define the custom C++ classes and how they interact.  
**PAJO, HARVEY**

#### **IV. Testing and Results (Optional)**

4.1. Test Cases (Provide 2-3 sample tests showing input data and expected/actual output.)  
**GONZALES, JAIRON**

4.2. Performance Test (Prove that NFR1 is met by testing with the 50+ record input.)  
**PAJO, HARVEY**

#### **V. Conclusion and Contributions**

5.1. Conclusion - **BERMIO, CARL**

#### **CODE:**

Users.h - User.cpp - **PAJO, HARVEY**

ReminderTime.h - Reminder.cpp - **GONZALES, JAIRON**

MedicineList.h - MedicineList.cpp - **MALDA, KENNETH**

MedicineReminder.h - MedicineReminder.cpp - **NEPA, JASTINE**

Main.cpp - **BERMIO, CARL**

GITHUB CREATION - **PAJO, HARVEY**

OVERALL PPT - **NEPA, JASTINE, MALDA, KENNETH**

OVERALL WORD - **PAJO, HARVEY**

#### ***References***

***Brown, M. T., & Bussell, J. K. (2011). Medication adherence: WHO cares? Mayo Clinic Proceedings, 86(4), 304–314. <https://doi.org/10.4065/mcp.2010.0575>***